RV32/RV64 Reference Card

Registers and Assembler Directives

Register	ABI Name	Description	Type
x0	zero	Zero constant	—
x1	ra	Return address	Temp.
x2	sp	Stack pointer	—
x3	gp	Global pointer	Saved
x4	tp	Thread pointer	Saved
x5-x7	t0-t2	Temporaries	Temp.
x8	s0 / fp	Saved / frame pointer	Saved
x9	s1	Saved register	Saved
x10-x11	a0-a1	Funct. arguments/return values	Temp.
x12-x17	a2-a7	Funct. arguments	Temp.
x18-x27	s2-s11	Saved registers	Saved
x28-x31	t3-t6	Temporaries	Temp.

* Under RISC-V conventional register use case, registers marked as Temp. can be modified at will by a called function; however, a call to a function cannot modify saved registers.

Register	ABI Name	Description	Type
f0-f7	ft0-ft7	FP temporaries	Temp.
f8-f9	fs0-fs1	FP saved registers	Saved
f10-f11	fa0-fa1	FP Funct. arguments/return values	Temp.
f12-f17	fa2-fa7	FP Funct. arguments	Temp.
f18-f27	fs2-fs11	FP saved registers	Saved
f28-f31	ft8-ft11	FP temporaries	Temp.

Assembler Directives:

- DECLARATION OF SEGMENTS:
  .data (for RW data) and .text (for program)
- DECLARATION OF INITIALIZED VARIABLES (WITHIN .data SEGMENT):
  .byte,.half,.word,.dword,.string – list of values/characters
- SPACE RESERVATION FOR VARIABLES (WITHIN .data SEGMENT):
  .zero – N bytes initialized with value 0

Instruction Formats

	31 27	26 25 24	20 19	15 14	12 11	7 6	0
R-Type	funct7		xb	xa	funct3	xd	opcode
R4-Type	xc	f2	xb	xa	funct3	xd	opcode
I-Type	$imm_{[11:0]}$			xa	funct3	xd	opcode
I2-Type	funct6	$imm_{[5:0]}$		xa	funct3	xd	opcode
S-Type	$imm_{[11:5]}$		xb	xa	funct3	$imm_{[4:0]}$	opcode
B-Type	$imm_{[12\mid10:5]}$		xb	xa	funct3	$imm_{[4:1\mid11]}$	opcode
U-Type	$imm_{[31:12]}$					xd	opcode
J-Type	$imm_{[20\mid10:1\mid11\mid19:12]}$					xd	opcode

RV32IM & RV64IM Instructions

	Instruction		Name	Encoding Type	Opcode	funct3	funct7	RTL Description
-	nop		No Operation	*Pseudo-inst.:* addi x0,x0,0				-
Move	li	xd,imm	Load (move) Immediate	*Pseudo-inst.:* Myriad sequence				xd ← imm
	la	xd,symbol	Load Address	*Pseudo-inst.:* auipc + addi				xd ← imm *(symbol)*
	lui	xd,imm	Load Upper Imm	U	0110111			xd ← imm << 12
	auipc	xd,imm	Add Upper Imm to PC	U	0010111			xd ← PC + (imm << 12)
	mv	xd,xs	Move Register Value	*Pseudo-inst.:* addi xd, xs, 0				xd ← xs
Memory	l{b|h|w|d}	xd,symbol	Load From Global Symbol	*Pseudo-inst.:* auipc + l{b|h|w|d}				xd ← M[symbol] (B/H/W/D)
	l{b|h|w|d}	xd,imm(xa)	Load {B, H, W, D}	I	0000011	{0,1,2,3}		xd ← M[xa+imm] (B/H/W/D)
	l{b|h|w}u	xd,imm(xa)	Load {B, H, W} (U)	I	0000011	{4,5,6}		xd ← M[xa+imm] (BU/HU/WU)
	s{b|h|w|d}	xs,symbol,xt	Store to Global Symbol	*Pseudo-inst.:* auipc + s{b|h|w|d}				M[symbol] ← xs *(modifies* xt*)*
	s{b|h|w|d}	xb,imm(xa)	Store {B,H,W,D}	S	0100011	{0,1,2,3}		M[xa+imm] ← xb (B/H/W/D)
Arithmetic, Logic, Shift	neg	xd,xs	Negate (2's complement)	*Pseudo-inst.:* sub xd, x0, xs				xd ← - xs
	add	xd,xa,xb	ADD	R	0110011	000	0000000	xd ← xa + xb
	addw	xd,xa,xb (RV64)	ADD Word	R	0111011	000	0000000	$xd_{[31:0]}$ ← $xa_{[31:0]}$ + $xb_{[31:0]}$
	addi	xd,xa,imm	ADD Immediate	I	0010011	000		xd ← xa + imm
	addiw	xd,xa,imm (RV64)	ADD Immediate Word	I	0011011	000		$xd_{[31:0]}$ ← $xa_{[31:0]}$ + imm
	sub	xd,xa,xb	SUB	R	0110011	000	0100000	xd ← xa - xb
	subw	xd,xa,xb (RV64)	SUB Word	R	0111011	000	0100000	$xd_{[31:0]}$ ← $xa_{[31:0]}$ - $xb_{[31:0]}$
	not	xd,xs	NOT	*Pseudo-inst.:* xori xd, xs, -1				xd ← ~ xs
	and	xd,xa,xb	AND	R	0110011	111	0000000	xd ← xa & xb
	andi	xd,xa,imm	AND Immediate	I	0010011	111		xd ← xa & imm
	andiw	xd,xa,imm (RV64)	AND Immediate Word	I	0011011	111		$xd_{[31:0]}$ ← $xa_{[31:0]}$ & imm
	or	xd,xa,xb	OR	R	0110011	110	0000000	xd ← xa | xb
	ori	xd,xa,imm	OR Immediate	I	0010011	110		xd ← xa | imm
	oriw	xd,xa,imm (RV64)	OR Immediate Word	I	0011011	110		$xd_{[31:0]}$ ← $xa_{[31:0]}$ | imm
	xor	xd,xa,xb	XOR	R	0110011	100	0000000	xd ← xa ^ xb
	xori	xd,xa,imm	XOR Immediate	I	0010011	100		xd ← xa ^ imm
	xoriw	xd,xa,imm (RV64)	XOR Immediate Word	I	0011011	100		$xd_{[31:0]}$ ← $xa_{[31:0]}$ ^ imm
	sll	xd,xa,xb	Shift Left Logical	R	0110011	001	0000000	xd ← xa << xb
	slli	xd,xa,imm	Shift Left Logical Imm	I2	0010011	001	000000‡	xd ← xa << imm
	slliw	xd,xa,imm (RV64)	Shift Left Logical Imm W	I2	0011011	001	000000‡	$xd_{[31:0]}$ ← $xa_{[31:0]}$ << imm
	srl	xd,xa,xb	Shift Right Logical	R	0110011	101	0000000	xd ← xa >> xb
	srli	xd,xa,imm	Shift Right Logical Imm	I2	0010011	101	000000‡	xd ← xa >> imm
	srliw	xd,xa,imm (RV64)	Shift Right Logical Imm W	I2	0011011	101	000000‡	$xd_{[31:0]}$ ← $xa_{[31:0]}$ >> imm
	sra	xd,xa,xb	Shift Right Arithmetic	R	0110011	101	0100000	xd ← xa >> xb
	srai	xd,xa,imm	Shift Right Arith Imm	I2	0010011	101	010000†	xd ← xa >> imm
	sraiw	xd,xa,imm (RV64)	Shift Right Arith Imm W	I2	0011011	101	010000†	$xd_{[31:0]}$ ← $xa_{[31:0]}$ >> imm

(RV64) Only available on RV64, i.e., cannot be used under RV32 mode.

‡ Shift operations with immediate (slli, srli and srai) use a 6-bit immediate to encode the shift ammount. The 6-th bit is stolen from the funct7 field. As a consequence, these operations use a funct6 field.

RV32IM & RV64IM Instructions (continued)

	Instruction		Name	Encoding Type	Opcode	funct3	funct7	RTL Description
Multiply, Divide	`mul`	`xd,xa,xb`	Multiply	R	0110011	000	0000001	xd ← (xa × xb) (low part)
	`mulw`	`xd,xa,xb` (RV64)	Multiply	R	0111011	000	0000001	xd ← (xa × xb) (low part)
	`mulh`	`xd,xa,xb`	Multiply High (S×S)	R	0110011	001	0000001	xd ← (xa × xb) (high part)
	`mulsu`	`xd,xa,xb`	Multiply High (S×U)	R	0110011	010	0000001	xd ← (xa × xb) (high part)
	`mulu`	`xd,xa,xb`	Multiply High (U×U)	R	0110011	011	0000001	xd ← (xa × xb) (high part)
	`div`	`xd,xa,xb`	Divide	R	0110011	100	0000001	xd ← xa / xb
	`divw`	`xd,xa,xb` (RV64)	Divide Word	R	0111011	100	0000001	xd ← xa / xb
	`divu`	`xd,xa,xb`	Divide (U)	R	0110011	101	0000001	xd ← xa / xb
	`divuw`	`xd,xa,xb` (RV64)	Divide (U) Word	R	0111011	101	0000001	xd ← xa / xb
	`rem`	`xd,xa,xb`	Remainder	R	0110011	110	0000001	xd ← xa % xb
	`remw`	`xd,xa,xb` (RV64)	Remainder Word	R	0111011	110	0000001	xd ← xa % xb
	`remu`	`xd,xa,xb`	Remainder (U)	R	0110011	111	0000001	xd ← xa % xb
	`remuw`	`xd,xa,xb` (RV64)	Remainder (U) Word	R	0111011	111	0000001	xd ← xa % xb
Compare	`slt`	`xd,xa,xb`	Set Less Than	R	0110011	010	0000000	xd ← (xa < xb)?1:0
	`slti`	`xd,xa,imm`	Set Less Than Imm	I	0010011	010		xd ← (xa < imm)?1:0
	`sltu`	`xd,xa,xb`	Set Less Than (U)	R	0110011	011	0000000	xd ← (xa < xb)?1:0
	`sltiu`	`xd,xa,imm`	Set Less Than Imm (U)	I	0010011	011		xd ← (xa < imm)?1:0
	`seqz`	`xd,xs`	Set Equal Zero	*Pseudo-inst.:* `sltiu xd, xs, 1`				xd ← (xs == 0)?1:0
	`snez`	`xd,xs`	Set Not Equal Zero	*Pseudo-inst.:* `sltu xd, x0, xs`				xd ← (xs != 0)?1:0
	`sltz`	`xd,xs`	Set Less Than Zero	*Pseudo-inst.:* `slt xd, xs, x0`				xd ← (xs < 0)?1:0
	`sgtz`	`xd,xs`	Set Greater Than Zero	*Pseudo-inst.:* `slt xd, x0, xs`				xd ← (xs > 0)?1:0
Flow control (branch, jump, call, ret)	`beq`	`xa,xb,imm`	Branch Equal	B	1100011	000		if(xa==xb) PC ← PC + imm
	`bne`	`xa,xb,imm`	Branch Not Equal	B	1100011	001		if(xa!=xb) PC ← PC + imm
	`bgt`	`xs, xt, imm`	Branch Greater Than	*Pseudo-inst.:* `blt xt, xs, imm`				if (xs> xt) PC ← symbol
	`bge`	`xa,xb,imm`	Branch Greater or Equal	B	1100011	101		if(xa>=xb) PC ← PC + imm
	`ble`	`xs, xt, imm`	Branch Less or Equal	*Pseudo-inst.:* `bge xt, xs, imm`				if (xs<=xt) PC ← symbol
	`blt`	`xa,xb,imm`	Branch Less Than	B	1100011	100		if(xa< xb) PC ← PC + imm
	`bgtu`	`xs, xt, imm`	Branch Less Than (U)	*Pseudo-inst.:* `bltu xt, xs, imm`				if (xs> xt) PC ← symbol
	`bgeu`	`xa,xb,imm`	Branch Greater or Equal (U)	B	1100011	111		if(xa>=xb) PC ← PC + imm
	`bleu`	`xs, xt, imm`	Branch Less or Equal (U)	*Pseudo-inst.:* `bgeu xt, xs, imm`				if (xs<=xt) PC ← symbol
	`bltu`	`xa,xb,imm`	Branch Less Than (U)	B	1100011	110		if(xa< xb) PC ← PC + imm
	`beqz`	`xs, symbol`	Branch Equal Zero	*Pseudo-inst.:* `beq xs, x0, imm`				if (xs==0) PC ← symbol
	`bnez`	`xs, symbol`	Branch Not Equal Zero	*Pseudo-inst.:* `bne xs, x0, imm`				if (xs!=0) PC ← symbol
	`blez`	`xs, symbol`	Branch Less or Equal Zero	*Pseudo-inst.:* `bge x0, xs, imm`				if (xs<=0) PC ← symbol
	`bgez`	`xs, symbol`	Branch Greater or Equal Zero	*Pseudo-inst.:* `bge xs, x0, imm`				if (xs>=0) PC ← symbol
	`bltz`	`xs, symbol`	Branch Less Than Zero	*Pseudo-inst.:* `blt xs, x0, imm`				if (xs< 0) PC ← symbol
	`bgtz`	`xs, symbol`	Branch Greater Than Zero	*Pseudo-inst.:* `blt x0, xs, imm`				if (xs> 0) PC ← symbol
	`j`	`symbol`	Jump	*Pseudo-inst.:* `jal x0, offset`				PC ← symbol
	`jal`	`xd,imm`	Jump And Link	J	1101111			xd ← PC+4; PC ← PC + imm
	`jal`	`symbol`	Jump And Link To Symbol	*Pseudo-inst.:* `jal x1, offset`				x1 ← PC + 4; PC ← symbol
	`jr`	`xs`	Jump Register	*Pseudo-inst.:* `jalr x0, xs, 0`				PC ← xs
	`jalr`	`xs`	Jump And Link Register	*Pseudo-inst.:* `jalr x1, xs, 0`				x1 ← PC + 4; PC ← xs
	`jalr`	`xd,xa,imm`	Jump And Link Register	I	1100111	000		xd ← PC+4; PC ← xa + imm
	`call`	`symbol`	Call subroutine	*Pseudo-inst.:* `auipc + jalr`				x1 ← PC + 4; PC ← symbol
	`ret`		Return from subroutine	*Pseudo-inst.:* `jalr x0, x1, 0`				PC ← x1
OS [I]	`ecall` *		Environment Call	I	1110011	imm=0x0, others=0		SEPC ← PC + 4; PC ← STVEC
	`ebreak`		Environment Break	I	1110011	imm=0x1, others=0		SEPC ← PC + 4; PC ← STVEC
	`sret`		Exception return	I	1110011	imm=0x102, others=0		PC ← SEPC

(RV64) Only available on RV64, i.e., cannot be used under RV32 mode. | * `ecall` also sets: `SPIE ← GIE; GIE ← 0`

RV32FD and RV64FD Floating Point Extensions (not available on Ripes)

	{"F"|"D"} Instruction		Name	Encoding Type	Opcode	funct3	funct7	RTL Description
LD, ST	`fl{w|d}`	`fd,symbol`	FP Load from Symbol	*Pseudo-inst.:* `auipc + fl{w|d}`				`fd ← M[symbol]`
	`fl{w|d}`	`fd,imm(xa)`	FP Load	I	0000111	{010|011}		`fd ← M[xa+imm]`
	`fs{w|d}`	`fb,symbol,xt`	FP Store to Symbol	*Pseudo-inst.:* `auipc + fs{w|d}`				`M[symbol] ← fb` *(modifies* `xt`*)*
	`fs{w|d}`	`fb,imm(xa)`	FP Store	S	0100111	{010|011}		`M[xa+imm] ← fb`
Move	`fmv.{s|d}`	`fd,fs`	FP Sign Injection	*Pseudo-inst.:* `fsgnj.{s|d} fd,fs,fs`				`fd = fs`
	`fsgnj.{s|d}`	`fd,fa,fb`	FP Sign Injection	R	1010011	000	{0x10|0x11}	`fd = abs(fa) * sgn(fb)`
	`fsgnjn.{s|d}`	`fd,fa,fb`	FP Sign Neg Injection	R	1010011	001	{0x10|0x11}	`fd = abs(fa) * -sgn(fb)`
	`fsgnjx.{s|d}`	`fd,fa,fb`	FP Sign Xor Injection	R	1010011	010	{0x10|0x11}	`fd = fa * sgn(fb)`
	`fabs.{s|d}`	`fd,fs`	FP Absolute Value	*Pseudo-inst.:* `fsgnjx.{s|d} fd,fs,fs`				`fd = |fs|`
	`fneg.{s|d}`	`fd,fs`	FP Negative Value	*Pseudo-inst.:* `fsgnjn.{s|d} fd,fs,fs`				`fd = -fs`
	`fmin.{s|d}`	`fd,fa,fb`	FP Minimum	R	1010011	000	{0x12|0x13}	`fd = min(fa, fb)`
	`fmax.{s|d}`	`fd,fa,fb`	FP Maximum	R	1010011	001	{0x12|0x13}	`fd = max(fa, fb)`
Arithmetic	`fadd.{s|d}`	`fd,fa,fb`	FP Add	R	1010011	rm	{0x00|0x01}	`fd = fa + fb`
	`fsub.{s|d}`	`fd,fa,fb`	FP Sub	R	1010011	rm	{0x04|0x05}	`fd = fa - fb`
	`fmul.{s|d}`	`fd,fa,fb`	FP Mul	R	1010011	rm	{0x08|0x09}	`fd = fa * fb`
	`fdiv.{s|d}`	`fd,fa,fb`	FP Div	R	1010011	rm	{0x0c|0x0d}	`fd = fa / fb`
	`fsqrt.{s|d}`	`fd,fa`	FP Square Root	R	1010011	rm	{0x2c|0x2d}	`fd = sqrt(fa)`
Fused	`fmadd.{s|d}`	`fd,fa,fb,fc`	FP Fused Mul-Add	R4	1000011	rm	f2={00,01}	`fd = fa * fb + fc`
	`fmsub.{s|d}`	`fd,fa,fb,fc`	FP Fused Mul-Sub	R4	1000111	rm	f2={00,01}	`fd = fa * fb - fc`
	`fnmadd.{s|d}`	`fd,fa,fb,fc`	FP Neg Fused Mul-Add	R4	1001111	rm	f2={00,01}	`fd = -fa * fb + fc`
	`fnmsub.{s|d}`	`fd,fa,fb,fc`	FP Neg Fused Mul-Sub	R4	1001011	rm	f2={00,01}	`fd = -fa * fb - fc`

`rm` – Floating point rounding mode. Set to "`000`" to select round to nearest.

RV32FD and RV64FD Floating Point Extensions (continued) (not available on Ripes)

	{"F"|"D"} Instruction	Name	Encoding Type	Opcode	funct3	funct7	RTL Description
Move/Convert	fcvt.{s|d}.w fd,xa	Convert from S. Int32	R	1010011	rm (rb=0)	{0x68|0x69}	fd = (float/double) xa
	fcvt.{s|d}.wu fd,xa	FP Conv from U. Int32	R	1010011	rm (rb=1)	{0x68|0x69}	fd = (float/double) xa
	fcvt.{s|d}.l fd,xa (RV64)	FP Conv from S. Int64	R	1010011	rm (rb=2)	{0x68|0x69}	fd = (float/double) xa
	fcvt.{s|d}.lu fd,xa (RV64)	FP Conv from U. Int64	R	1010011	rm (rb=3)	{0x68|0x69}	fd = (float/double) xa
	fcvt.w.{s|d} xd,fa	FP Convert to S. Int32	R	1010011	rm (rb=0)	{0x60|0x61}	xd = (int32_t) fa
	fcvt.wu.{s|d} xd,fa	FP Convert to U. Int32	R	1010011	rm (rb=1)	{0x60|0x61}	xd = (uint32_t) fa
	fcvt.l.{s|d} xd,fa (RV64)	FP Convert to S. Int64	R	1010011	rm (rb=2)	{0x60|0x61}	xd = (int64_t) fa
	fcvt.lu.{s|d} xd,fa (RV64)	FP Convert to U. Int64	R	1010011	rm (rb=3)	{0x60|0x61}	xd = (uint64_t) fa
	fcvt.s.d fd,fa	DP FP Convert to SP FP	R	1010011	rm (rb=0)	0x40	fd = (float) fa
	fcvt.d.s fd,fa	SP FP Convert to DP FP	R	1010011	rm (rb=1)	0x41	fd = (double) fa
	fmv.x.w xd,fa	Move SP FP to Int32	R	1010011	000	0x70	xd = fa (32-bit move)
	fmv.w.x fd,xa	Move Int32 to SP FP	R	1010011	000	0x78	fd = xa (32-bit move)
	fmv.x.d xd,fa (RV64)	Move SP FP to Int32	R	1010011	000	0x71	xd = fa (64-bit move)
	fmv.d.x fd,xa (RV64)	Move Int32 to SP FP	R	1010011	000	0x79	fd = xa (64-bit move)
Compare	feq.{s|d} xd,fa,fb	FP Equal	R	1010011	010	{0x50|0x51}	xd = (fa == fb) ? 1 : 0
	flt.{s|d} xd,fa,fb	FP Less Than	R	1010011	001	{0x50|0x51}	xd = (fa < fb) ? 1 : 0
	fle.{s|d} xd,fa,fb	FP Less or Equal	R	1010011	000	{0x50|0x51}	xd = (fa <= fb) ? 1 : 0
	fclass.{s|d} xd,fa	FP Classify	R	1010011	001	{0x70|0x71}	xd = (fa type?):0..9

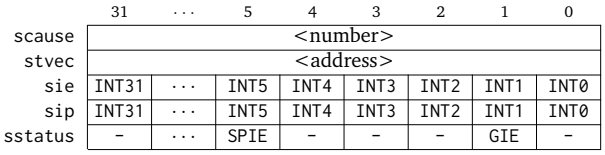
rm – Floating point rounding mode. Set to "000" to select round to nearest.

Observation: Designations fl{w|d} and fs{w|d} represent the corresponding instructions for SP FP load/store (flw/fsw) and for DP FP load/store (fld/fsd). Similarly, designations f__.{s|d} are used to represent the corresponding SP (f__.s) and DP (f__.d) instructions.

Special purpose registers and instructions (not available on Ripes)

Relevant Control and Status Registers (CSRs):

Type	Register	Full Name	CSR ID
Exception handling	sepc	Supervisor Exception PC	0x141
	scause	Supervisor Exception Cause	0x142
	stvec	Supervisor Trap Vector Base Register	0x105
	sip	Supervisor Interrupt-Pending Register	0x144
	sie	Supervisor Interrupt-Enable Register	0x104
	sstatus	Supervisor Status Register	0x100
Virtual Memory	satp	Supervisor Address Translation and Protection	0x180



states the number of the first pending interruption
states the address of the Interrupt Service Routine
0 - Interruptions masked; 1 - Interruptions enabled
0 - No pending interruption; 1 - Pending interruption

GIE - Global Interrupt Enable (set to zero to disable ALL interruptions/exceptions). When entering an interrupt service routine (ISR), the value of GIE is first copied to SPIE and then set to zero. When leaving the ISR, its value is restored using SPIE.

SPIE - GIE backup copy. The value of GIE is automatically copied to SPIE when entering the interrupt service routine.

	Instruction	Name	Encoding Type	Opcode	funct3	funct5	RTL Description
Handling of CSR registers	csrrc xd,csr,xa *	CSR read and clear	I	1110011	011		xd ← CSR[csr]; CSR[csr] ← CSR[csr] & ∼xa
	csrrci xd,csr,imm2 *	CSR read and clear	I	1110011	111		xd ← CSR[csr]; CSR[csr] ← CSR[csr] & ∼imm2
	csrrs xd,csr,xa *	CSR read and set	I	1110011	010		xd ← CSR[csr]; CSR[csr] ← CSR[csr] | xa
	csrrsi xd,csr,imm2 *	CSR read and set	I	1110011	110		xd ← CSR[csr]; CSR[csr] ← CSR[csr] | imm2
	csrrw xd,csr,xa *	CSR read and write	I	1110011	001		xd ← CSR[csr]; CSR[csr] ← xa
	csrrwi xd,csr,imm2 *	CSR read and write	I	1110011	101		xd ← CSR[csr]; CSR[csr] ← imm2
	csrr xd,csr	CSR read	*Pseudo-inst.:* csrrs rd,csr,x0				xd ← CSR[csr]
	csrw csr,xa	CSR write	*Pseudo-inst.:* csrrw x0,csr,xa				CSR[csr] ← xa
	csrc csr,xa	CSR clear bits	*Pseudo-inst.:* csrrw x0,csr,xa				CSR[csr] ← CSR[csr] & ∼xa
	csrs csr,xa	CSR set bits	*Pseudo-inst.:* csrrs x0,csr,xa				CSR[csr] ← CSR[csr] | xa
	csrwi csr,imm2	CSR write	*Pseudo-inst.:* csrrwi x0,csr,imm2				CSR[csr] ← imm2
	csrci csr,imm2	CSR clear bits	*Pseudo-inst.:* csrrwi x0,csr,imm2				CSR[csr] ← CSR[csr] & ∼imm2
	csrsi csr,imm2	CSR set bits	*Pseudo-inst.:* csrrsi x0,csr,imm2				CSR[csr] ← CSR[csr] | imm2

∗ – To encode the instructions use field imm to specify csr and ra to specify imm2.